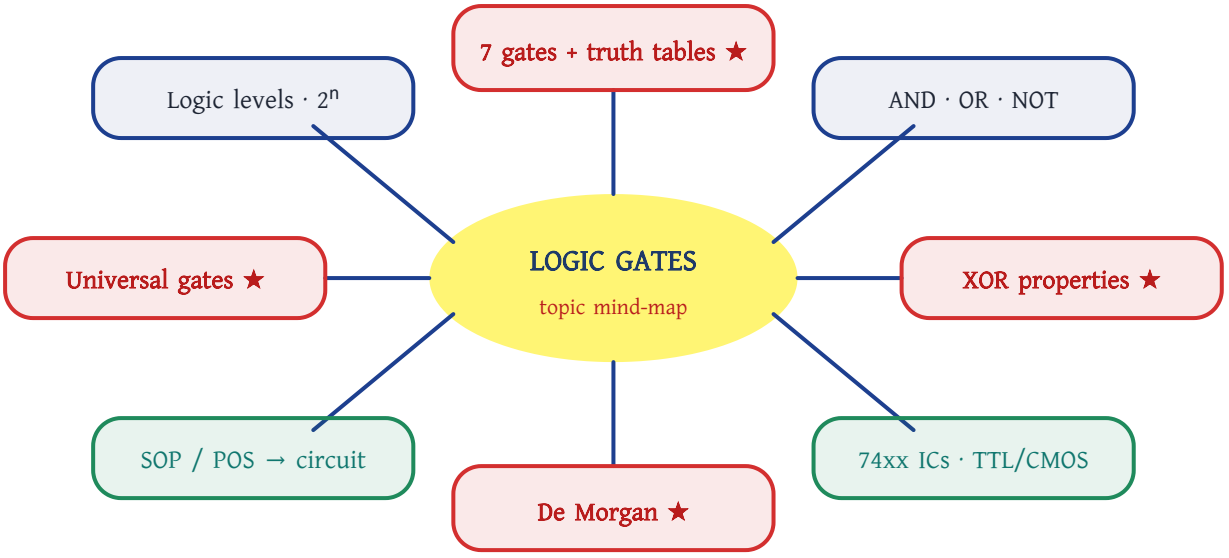
Logic Gates

Truth tables · universal gates · De Morgan · ICs: exam-ready in 10 sheets



↑ the whole topic on one page: red = high-yield, green = applied

High-yield list

① 2-input truth tables
② min. NAND/NOR counts
③ De Morgan & bubbles
④ XOR = odd parity
⑤ 7400/02/04/08/32/86
⑥ +ve vs -ve logic

Syllabus coverage

"Digital logic: Boolean algebra, **De Morgan's Theorems**, **logic gates and their truth tables**, canonical forms, combinational logic circuits, minimization techniques". This notebook covers the gate part in depth and bridges into canonical forms; K-maps are in the Boolean Algebra lesson.

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ Fundamentals & logic levels ..... p. 2
✓ AND, OR, NOT ..... p. 3
✓ NAND, NOR, XOR, XNOR ..... p. 4
✓ Universal gates & counts ..... p. 5
✓ De Morgan & bubble pushing ..... p. 6
✓ Expression → circuit (SOP/POS) ... p. 7
✓ ICs, families, applications ..... p. 8
✓ Night-before sheet ..... p. 9
✓ Practice MCQs + written ..... p. 10

1 What Is a Logic Gate?

Definition: A logic gate is an electronic circuit that takes one or more **binary inputs** and produces one binary output according to a fixed **Boolean function**. Gates are the building blocks of every digital system: ALU, memory, CPU control.

> Gate = Boolean operator made of transistors

Boolean operation	Symbol	Gate	Output is 1 when ...
Logical product	A·B	AND	all inputs are 1
Logical sum	A+B	OR	at least one input is 1
Complement	A′	NOT	the input is 0

Counting rules:

n inputs → 2^n rows in the truth table

n variables → $2^{(2^n)}$ different Boolean functions

(n = 2 → 4 rows, **16** functions)

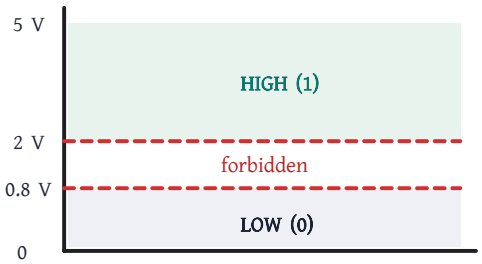
Memory trick: Write the input columns like binary counting: 00, 01, 10, 11. You can never miss a row.

2 Logic Levels: Positive vs Negative Logic

A gate does not "see" 0 and 1; it sees **voltage ranges**.

- **Positive logic**: HIGH = 1, LOW = 0 (the default)
- **Negative logic**: HIGH = 0, LOW = 1

Same circuit, different labels → a different function.



TTL input thresholds

TTL (5 V) level	Input	Output
LOW (0)	V_{IL} ≤ **0.8 V**	V_{OL} ≤ 0.4 V
HIGH (1)	V_{IH} ≥ **2.0 V**	V_{OH} ≥ 2.4 V

Noise margin = 2.4 − 2.0 = 0.8 − 0.4 = **0.4 V**. CMOS thresholds scale with the supply (roughly 30% / 70% of V_{DD}).

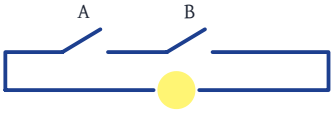
Exam trap: A positive-logic **AND** gate behaves as an **OR** gate in negative logic; positive **NAND** = negative **NOR** (De Morgan in disguise).

3 The Basic Gates: AND, OR, NOT

AND: output 1 only if ALL inputs are 1 · Y = A·B

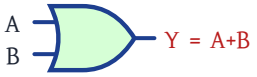


A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

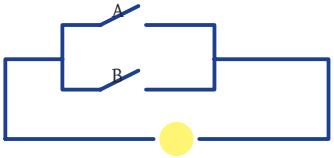


switches in series

OR: output 0 only if ALL inputs are 0 · Y = A + B



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1



switches in parallel

NOT (inverter): output = complement · Y = A′



A	Y
0	1
1	0

The only **single-input** gate. The bubble ○ on any symbol means NOT. Two NOTs in a row cancel: (A′)′ = A.

Multi-input gates

- n-input AND: 1 only for 11...1 (one row out of 2^n)
- n-input OR: 0 only for 00...0
- AND, OR are associative: (A·B)·C = A·(B·C), so 2-input gates can be **cascaded**

Gating (enable / inhibit):

AND: control 1 → **passes** data, 0 → forces 0

OR: control 0 → passes data, 1 → forces 1

XOR: control 1 → **inverts** data

Memory trick: **AND = "all"**, **OR = "any"**. AND looks for one 0 to kill the output; OR looks for one 1 to set it.

4 NAND, NOR, XOR, XNOR

NAND = NOT + AND:



Y = (AB)′ · 0 **only** when all inputs are 1

NOR = NOT + OR:



Y = (A+B)′ · 1 **only** when all inputs are 0

XOR (exclusive-OR):



A⊕B = **A′B + AB′** · 1 when inputs **differ**

XNOR (equivalence):



A⊙B = **AB + A′B′** · 1 when inputs are **equal**

Master truth table (all 2-input gates)

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

Complement pairs: AND↔NAND, OR↔NOR, XOR↔XNOR

XOR properties:

$A \oplus 0 = A$ (pass)

$A \oplus 1 = \mathbf{A'}$ (invert)

$A \oplus A = 0$

$A \oplus A' = 1$

commutative & associative

$(A \oplus B)' = A' \oplus B = A \oplus B'$

n-input XOR = 1 when the number of 1s is **odd** → **odd-parity function**; used for parity generators/checkers, comparators, adders, controlled inverters.

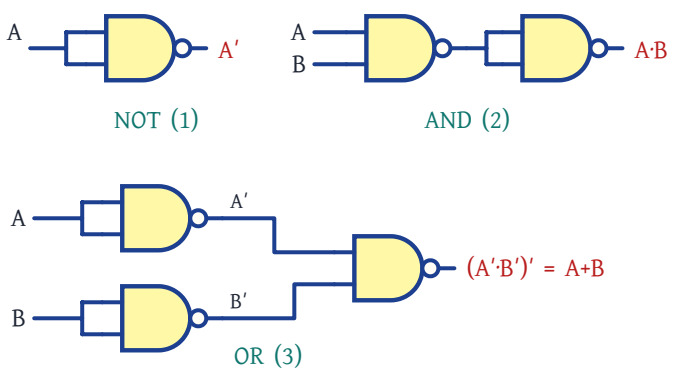
Exam trap: NAND and NOR are **not associative**: (A NAND B) NAND C ≠ A NAND (B NAND C). A 3-input NAND is (ABC)′, not a cascade of two 2-input NANDs.

Memory trick: "N" in front = put a bubble on the output = flip that gate's column. XOR = "**different** → 1", XNOR = "**same** → 1".

5 Universal Gates: NAND & NOR

Definition: A gate is **universal** if NOT, AND and OR (hence **any** Boolean function) can be built from it alone. **NAND** and **NOR** are universal; AND, OR, XOR are not.

Only NAND



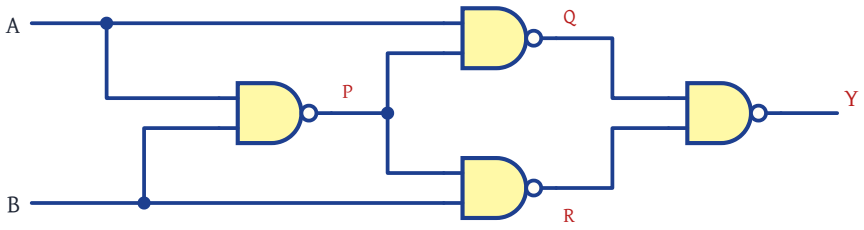
tie inputs together:
NAND(A, A) = A′

Only NOR (mirror image): NOR(A, A) = A′ (1 gate) · NOR then NOR-inverter = **OR** (2) · invert both inputs, then NOR: (A′ + B′)′ = **AND** (3).

Function	NOT	AND	OR	XOR	XNOR
Min. 2-input **NAND** gates	1	2	3	**4**	5
Min. 2-input **NOR** gates	1	3	2	5	**4**

> NAND and NOR swap the AND/OR and XOR/XNOR counts (duality)

XOR with 4 NAND gates (the minimum)



① $P = (AB)' = A' + B'$

② $Q = (A \cdot P)' = (A(A'+B'))' = (AB')'$

③ $R = (B \cdot P)' = (A'B)'$

④ $Y = (Q \cdot R)' = AB' + A'B = \mathbf{A \oplus B}$

Exam trap: Many textbooks draw XOR with **5 NAND** or **6 NOR** (inverters + SOP). Those work but are not minimal; MCQs ask the **minimum: 4 NAND, 5 NOR.**

6 De Morgan's Theorems

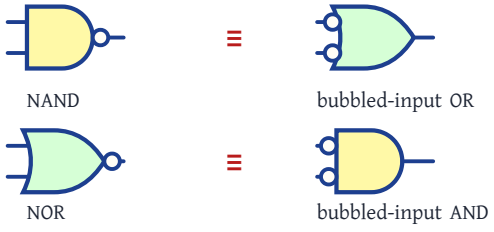
The two laws: $\mathbf{(A \cdot B)' = A' + B'}$ $\mathbf{(A + B)' = A' \cdot B'}$

n variables: $(A \cdot B \cdot C \ldots)' = A' + B' + C' + \ldots$ · $(A + B + C + \ldots)' = A' \cdot B' \cdot C' \ldots$

Proof by truth table

A	B	(AB)'	A'+B'	(A+B)'	A'B'
0	0	1	1	1	1
0	1	1	1	0	0
1	0	1	1	0	0
1	1	0	0	0	0

Equivalent symbols



Memory trick: "**Break the bar, change the sign**". In symbols: **bubble pushing**: move bubbles from output to all inputs (or back) and swap AND ↔ OR.

Worked example (written-answer format)

Q: Simplify F = (A + B)′ + (A′ + B′)′ and name the gate it equals. 4 marks

1. De Morgan on each term: (A + B)′ = A′B′ and (A′ + B′)′ = A″B″ = AB
2. So F = A′B′ + AB
3. This is 1 when A = B → F = **A ⊙ B (XNOR)**
4. Circuit: one XNOR gate replaces 2 NOR + 2 NOT + 1 OR → **fewer gates, lower cost & delay**

7 Duality Principle

Rule: Every Boolean identity stays true if you swap **+** ↔ **·** and **0** ↔ **1** (variables are **not** complemented).

Identity	Its dual
A + 0 = A	A · 1 = A
A + A′ = 1	A · A′ = 0
A(B + C) = AB + AC	A + BC = (A + B)(A + C)
(A + B)′ = A′B′	(AB)′ = A′ + B′

Exam trap: The dual is **not** the complement. Complement also flips every variable (A → A′); the dual does not.

8 From Truth Table to Gate Circuit

Minterm (m): AND term with every variable, = 1 for exactly one row. **SOP** = OR of minterms = **Σm**

Maxterm (M): OR term with every variable, = 0 for exactly one row. **POS** = AND of maxterms = **ΠM**

1. Write the truth table (2^n rows)
2. Rows with F = 1 → minterms (SOP)
3. Simplify (Boolean laws or K-map)
4. Draw AND-OR (or NAND-NAND)
5. Check every row again

Worked example: 3-input majority voter

F = 1 when **at least two** of A, B, C are 1 (e.g. 2-out-of-3 approval).

#	A	B	C	F
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

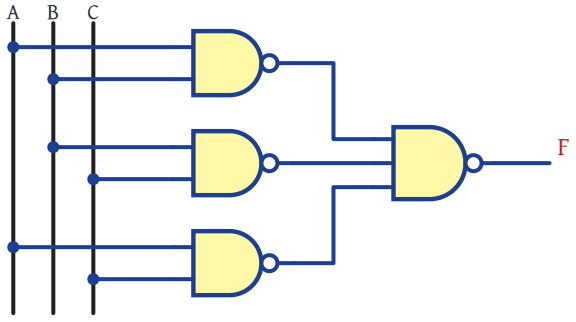
Canonical SOP: F = Σm(3, 5, 6, 7) = A′BC + AB′C + ABC′ + ABC

Canonical POS: F = ΠM(0, 1, 2, 4)

Simplify (add ABC twice, A + A′ = 1):

F = AB + BC + CA

= (A + B)(B + C)(C + A) in POS form



NAND-NAND: 3 two-input + 1 three-input NAND

Two-level rules:

SOP (AND-OR) → replace every gate by **NAND** = **NAND-NAND**

POS (OR-AND) → replace every gate by **NOR** = **NOR-NOR**

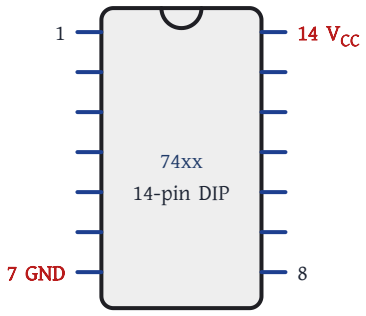
Why: F = ((AB)′(BC)′(CA)′)′ = AB + BC + CA by De Morgan

Fewer literals → fewer gates. Systematic minimisation (K-map, don't-cares) → see the **Boolean Algebra** lesson.

Exam trap: The minterm list and the maxterm list are **complements**: together they cover all 2^n row numbers (here {3,5,6,7} ∪ {0,1,2,4}).

9 Gate ICs (74xx TTL family)

IC	Contents
7400	quad 2-input NAND
7402	quad 2-input NOR
7404	hex inverter (6 NOT)
7408	quad 2-input AND
7432	quad 2-input OR
7486	quad 2-input XOR
74266	quad 2-input XNOR (open collector)



notch up, pin 1 top-left, count anticlockwise

"Quad" = 4 gates, "hex" = 6 gates. Standard 14-pin DIP: **V_{CC} = pin 14, GND = pin 7.**

10 Logic Families & Gate Parameters

	TTL (74xx)	CMOS (4000 / 74HC)
built from	bipolar transistors	MOSFETs (n + p pairs)
supply	5 V ± 5%	3–15 V (4000), 2–6 V (74HC)
static power	higher (mW per gate)	**very low** → used in CPUs, phones
fan-out	about 10 (standard TTL)	large (gate inputs draw ~no current)

- **Fan-in** = number of inputs of a gate · **Fan-out** = number of gate inputs one output can drive
- **Propagation delay** = input-to-output time (about 10 ns for standard TTL) → limits clock speed
- **Noise margin** = how much noise a level tolerates (TTL: 0.4 V)
- Integration scale (typical convention): **SSI** < 12 gates · **MSI** 12–99 · **LSI** 100–9,999 · **VLSI** ≥ 10,000

11 Where Gates Work

Classic circuits:

Half adder: S = A ⊕ B, C = AB

Even-parity bit for A, B, C: P = A ⊕ B ⊕ C

Equality (1-bit comparator): A ⊙ B

IT link: C bitwise:

12 & 10 = **8** · 12 | 10 = **14** · 12 ^ 10 = **6** · ~12 = −13

Masks: x & 1 → odd/even test · x | mask sets bits · x ^ mask toggles

Swap: a ^= b; b ^= a; a ^= b;

Memory trick: 74 + gate code: **00** NAND, **02** NOR, **04** NOT, **08** AND, **32** OR, **86** XOR. "Zero-zero is the universal hero".

Night-Before-the-Exam Sheet

Fact sheet:

AND: A·B · OR: A+B · NOT: A′

NAND: (AB)′ · NOR: (A+B)′

XOR: A′B + AB′ (differ → 1)

XNOR: AB + A′B′ (same → 1)

(AB)′ = A′+B′ · (A+B)′ = A′B′

A⊕0 = A · A⊕1 = A′ · A⊕A = 0

n inputs → 2^n rows · $2^{(2^n)}$ functions

Universal: NAND, NOR only

NAND: NOT 1, AND 2, OR 3, XOR 4

NOR: NOT 1, OR 2, AND 3, XNOR 4

SOP → NAND-NAND · POS → NOR-NOR

TTL: $V_{IL} \leq 0.8$ V, $V_{IH} \geq 2$ V, NM 0.4 V

7400 NAND · 7402 NOR · 7404 NOT

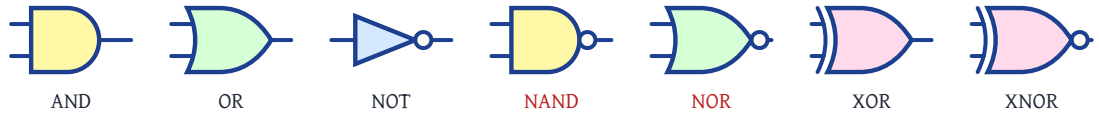
7408 AND · 7432 OR · 7486 XOR

Top 12 one-liners for MCQs

1. Basic gates: AND, OR, NOT
2. Universal gates: NAND, NOR
3. Only single-input gate: NOT
4. Output 1 for all-0 inputs: NAND, NOR, XNOR
5. Output 0 for all-1 inputs: NAND, NOR, XOR
6. XOR is also called "inequality" gate
7. XNOR is the "equality / equivalence" gate
8. NAND(A, A) = NOR(A, A) = A′
9. Min. NAND for XOR = 4
10. Positive AND = negative OR
11. Half-adder sum = XOR, carry = AND
12. 7404 = hex inverter (6 NOTs)

Top traps: OR is 0 **only** for all-0 inputs · NAND/NOR not associative · dual ≠ complement · XOR, XNOR and AND, OR are **not** universal · textbook 5-NAND XOR is not the minimum

All tricks together: AND = all, OR = any · "N" = bubble = flip the column · break the bar, change the sign · bubble pushing swaps AND ↔ OR · XOR = different, XNOR = same · 7400 = universal hero



draw all 7 symbols from memory before the exam

✓ Practice MCQs (exam pattern, 10 marks)

1. Which gate gives output 1 only when all inputs are 0?

(a) NAND
(b) NOR
(c) XOR
(d) OR

2. Minimum number of 2-input NAND gates to build XOR:

(a) 3
(b) 5
(c) 4
(d) 6

3. A ⊕ 1 equals:

(a) A
(b) 1
(c) 0
(d) A′

4. Rows in the truth table of a 4-input gate:

(a) 8
(b) 16
(c) 4
(d) 32

5. IC 7404 contains:

(a) 6 inverters
(b) 4 NAND
(c) 4 NOR
(d) 4 XOR

6. A positive-logic AND gate works in negative logic as:

(a) NAND
(b) NOR
(c) OR
(d) AND

7. $(A + B)'$ equals:

(a) $A'B'$

(b) $A' + B'$

(c) AB

(d) $(AB)'$

8. Number of distinct Boolean functions of 2 variables:

(a) 4

(b) 8

(c) 2

(d) 16

9. Minimum 2-input NOR gates for an AND gate:

(a) 2

(b) 3

(c) 1

(d) 4

10. In C, 12 ^ 10 evaluates to:

(a) 6

(b) 8

(c) 14

(d) 2

Answer key → 1-b · 2-c · 3-d · 4-b · 5-a · 6-c · 7-a · 8-d · 9-b · 10-a

Written question: Implement a 2-input XOR gate using the minimum number of NAND gates only. Draw the circuit and prove it algebraically. 8 marks

1. **Circuit:** G1 = NAND(A, B) → P; G2 = NAND(A, P) → Q; G3 = NAND(B, P) → R; G4 = NAND(Q, R) → Y (diagram on sheet 5). 2
2. $P = (AB)' = A' + B'$ (De Morgan); $Q = (A(A' + B'))' = (AA' + AB')' = (AB')'$; $R = (A'B)'$ 3
3. $Y = (Q \cdot R)' = Q' + R' = \mathbf{AB' + A'B = A \oplus B}$ ✓ (De Morgan + double complement) 2
4. Check: AB = 00 → P = 1, Q = 1, R = 1, Y = 0 ✓; AB = 10 → P = 1, Q = 0, Y = 1 ✓. **4 gates = minimum.** 1

Before the exam ✓

- ✓ Can draw all 7 symbols + truth tables
- ✓ Can build NOT/AND/OR from NAND and NOR
- ✓ Know minimum gate counts (4 NAND XOR)
- ✓ Can apply De Morgan & bubble pushing
- ✓ Know 7400 … 7486 and V_{CC}/GND pins

> **Also practise (written):**
> ① XNOR from 4 NOR gates
> ② Majority voter with NAND only
> ③ Prove De Morgan by truth table
> ④ Even-parity generator for 4 bits